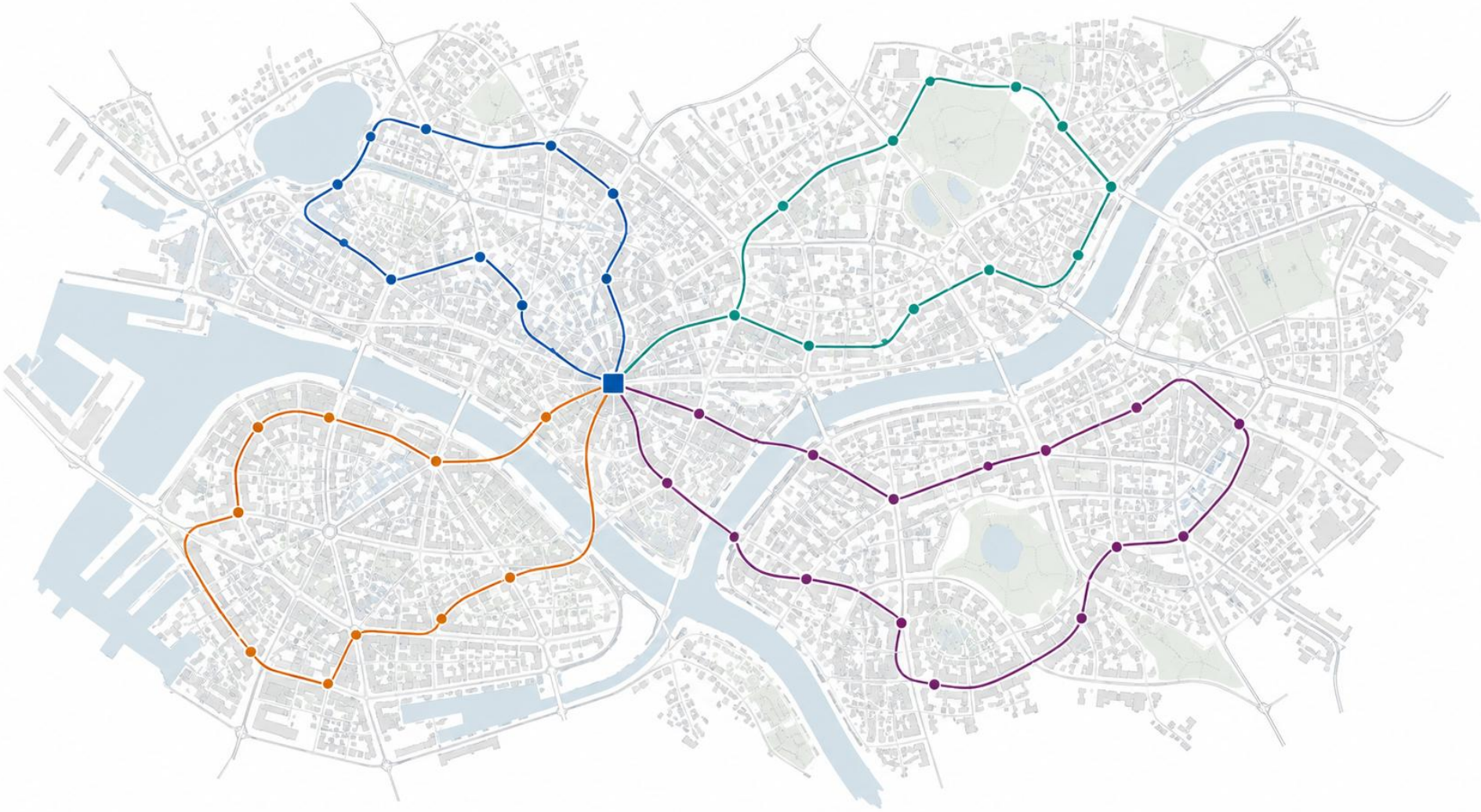
RL & Optimization



Motivation

Why ML

ML solves an optimization problem

$$\mathcal{L}(\theta) = \frac{1}{N} \sum_{i=1}^{N} \ell(f_\theta(x_i), y_i) + \lambda \, \Omega(\theta)$$

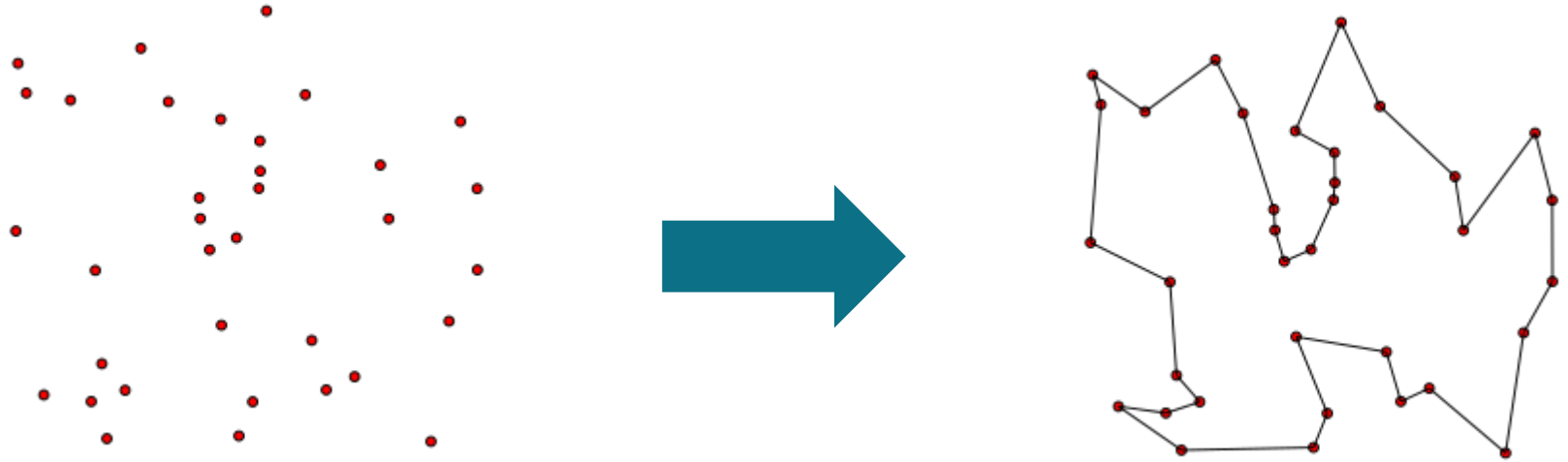
Now invert the question

Given an optimization problem, can we learn to solve it?

Traveling Salesman Problem (TSP)

Why ML

- Choose order of visits
- Goal: minimize distance
- NP-hard problem



Why bring learning into a solver?

Why ML | **Why SL**

Classical Algorithms

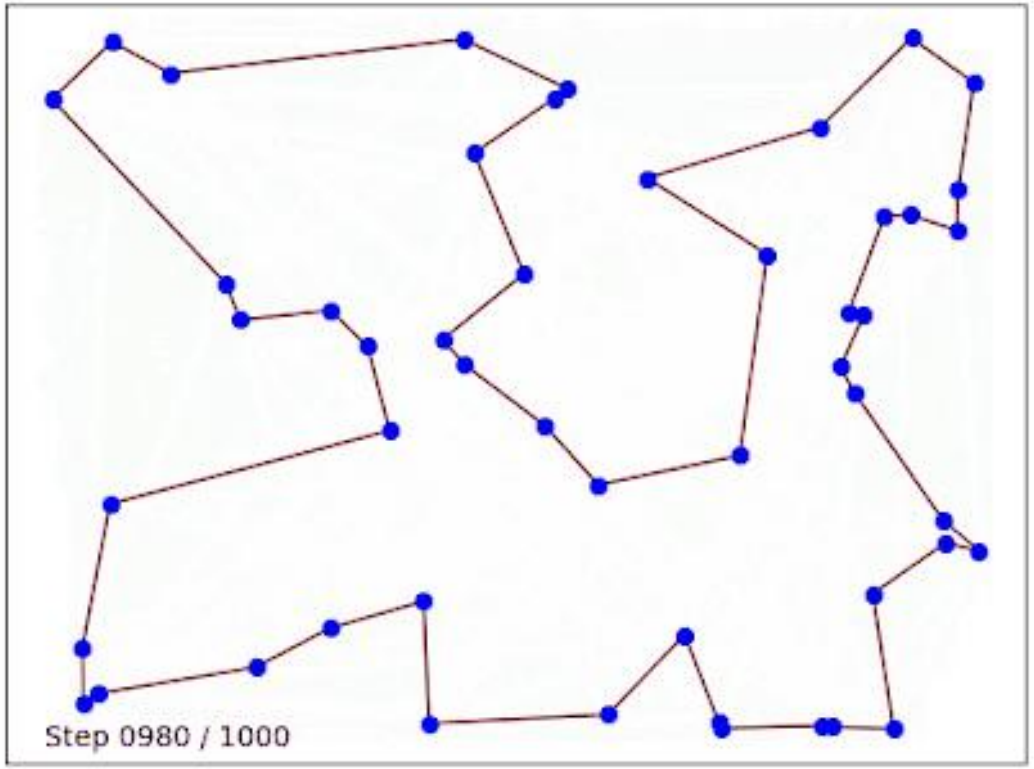
Genetic algorithms, local search… - SOTA
Solve each problem from scratch.

Supervised learning

Learn from solver-generated examples.

Generative models

Energy-based and diffusion models propose candidates; decoding enforces a tour.



DIFUSCO · Gaussian edge diffusion · Sun & Yang, NeurIPS 2023

Why optimize the policy with RL?

Why ML | Why SL | **Why RL**

- Imitation learns from a teacher. Exceeding it needs additional exploration.
- labelled solutions can be costly to obtain.

Why RL can help

- No labels
- Optimize route cost directly - improvement over a teacher is possible.
- Match the sequential nature of route construction.
- Adapt variants through the environment

What makes RL expensive

- Data-hungry and compute-intensive.
- Long-horizon credit assignment and high-variance gradients complicate learning.
- Training can be unstable.

Which RL approach

Value-based approach

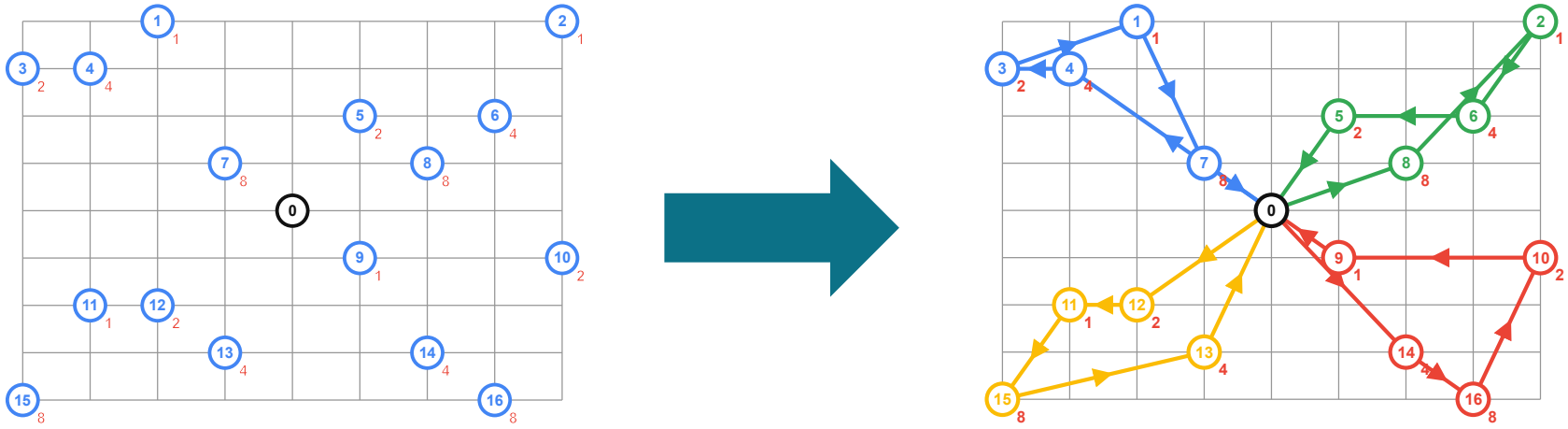
- Learning an exact optimal value function = Solve an NP hard problem in polynomial-time.
- Q-value-based approaches are sensitive to argmax instability when action values are close.
- Small value errors along a long trajectory can compound and destabilize training.

Policy-gradient approach

- Learn the next decision directly
- May still need value approximation (critic)
  - Critic-free workaround: a sampled (group-relative) rollout baseline (GRPO)
  - Other workarounds…

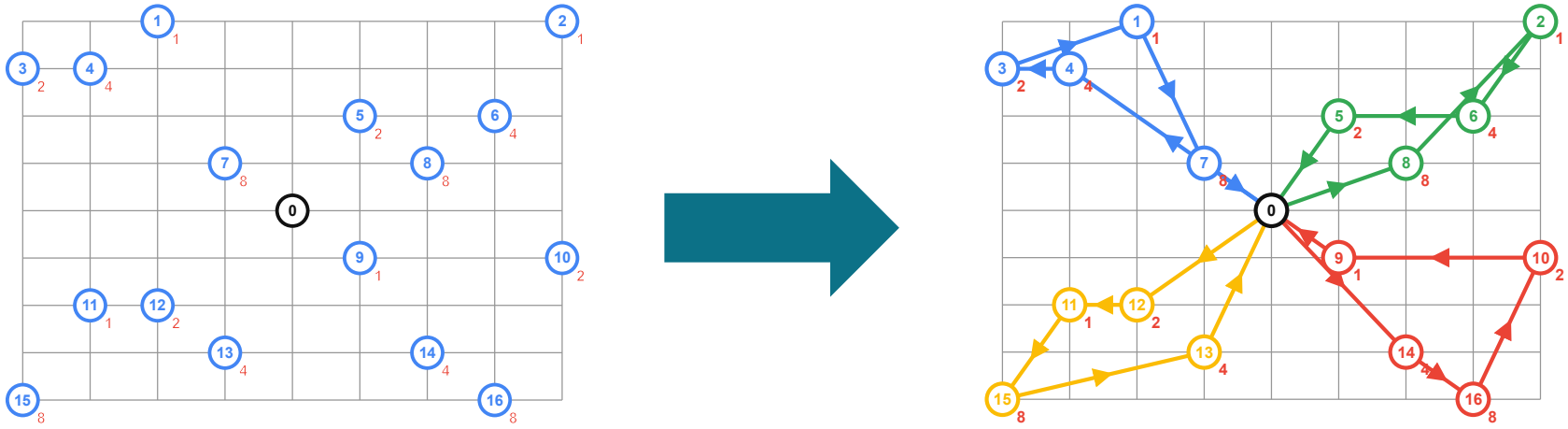
Capacitated Vehicle Routing Problem (CVRP)

- TSP + limited vehicle capacity
  - Need multiple vehicles
  - Goal: first minimize #vehicles, then minimize distance

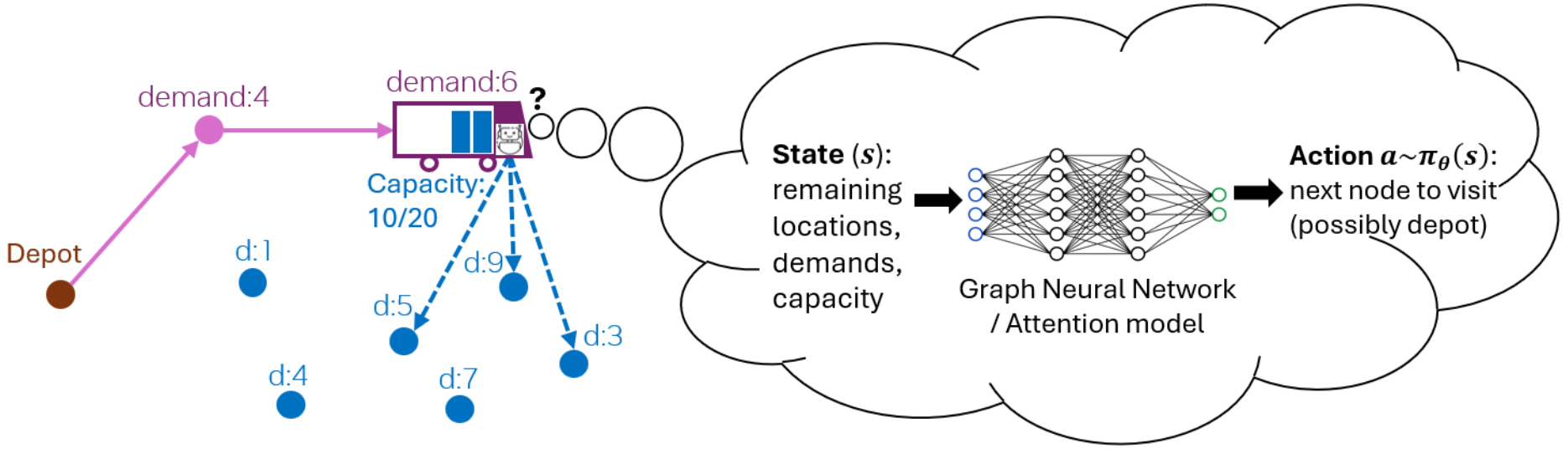


Vehicle Routing Problem (VRP)

- TSP + limited vehicle capacity
- U.S. freight transportation costs > $1,000B
  - 1% saving is worth over $10B
- Addressed by [NVIDIA's cuOpt]



RL for VRP: · Problem Formulation

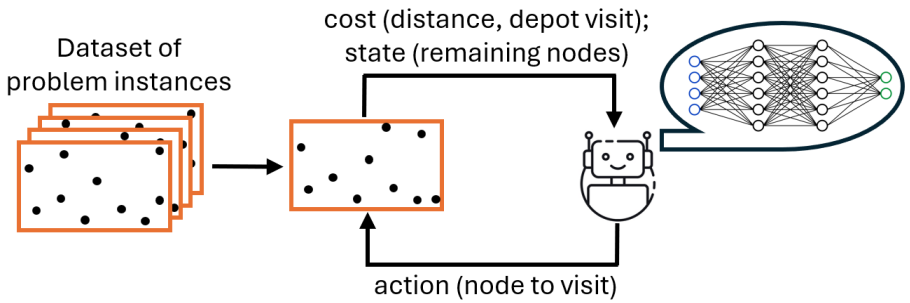


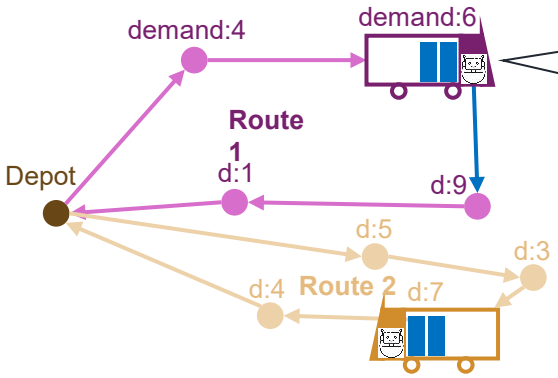
- Reward:
  - – Distance traveled
  - – Penalty on every new vehicle (depot visit)

Nazari et al., *Reinforcement Learning for Solving the Vehicle Routing Problem*, NeurIPS 2018

RL for VRP: · Algorithm

- Data:
  - Generate problem instances for training
  - Test on new instances
- Model: Graph Neural Network / Attention Model
- RL algorithm: Policy Gradient (REINFORCE / PPO)





Model update: Policy Gradient

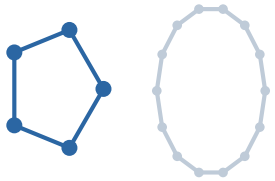
$$\theta_{k+1} \leftarrow \theta_k + \alpha \cdot \nabla \log \pi_{\theta_k}(s,a) \cdot \left(R_{t:T} - V_{\phi_k}(s)\right)$$

learning rate — probability to choose a — Return from s — Predicted return from s

- $R > V(s) \Rightarrow a$ was beneficial $\Rightarrow$ increase $\pi_\theta(s,a)$ next time

Does a trained RL model beat a SOTA solver?

No. But sometimes it does.



Smaller problems

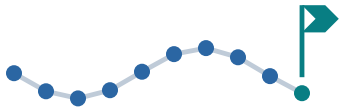


Shorter runtime

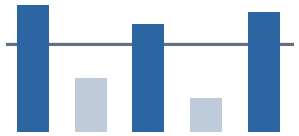


Exotic setups

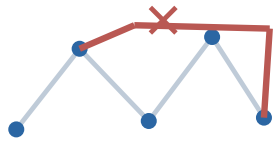
Why?



Long trajectories (Reward association problem)



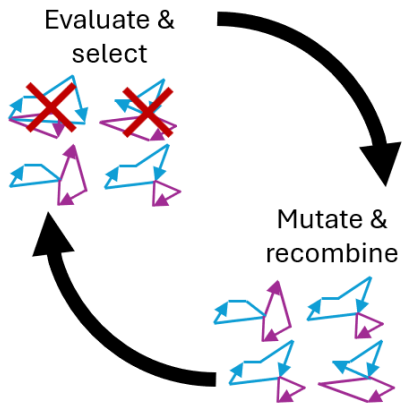
Poor critic / baseline & Small gaps



A single decision can ruin a whole solution

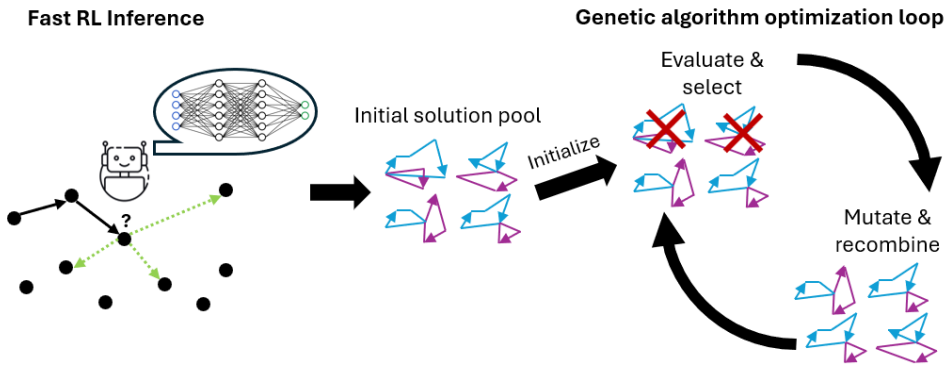
RL for VRP: · How to actually improve SOTA?

- Current SOTA: Genetic Algorithms (GA)
  - Start with random solutions and improve them iteratively
  - Solve every instance *from scratch*



RL for VRP: · How to actually improve SOTA?

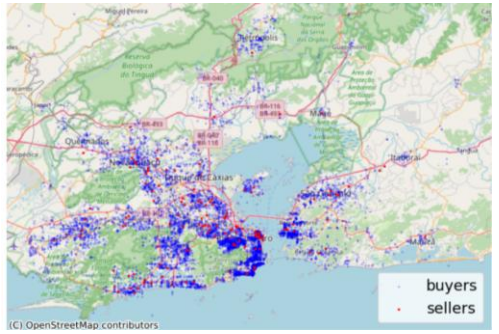
- Current SOTA: Genetic Algorithms (GA)
  - Start with random solutions and improve them iteratively
  - Solve every instance *from scratch*
- ***EARLI***: *Evolutionary Algorithm with RL Initialization*
  - Use RL to generate initial solutions (leverage data)
  - Use GA to improve solutions (leverage compute)



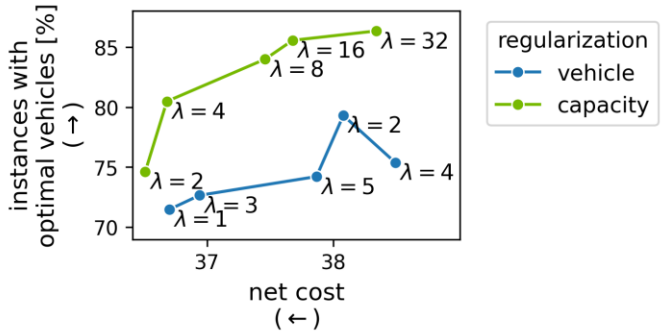
Greenberg et al., *Accelerating Vehicle Routing via AI-Initialized Genetic Algorithms*, 2025

RL for VRP: · Challenges we met

- On-policy RL: how to simulate data?
  - Simulation is different from real problems
  - Solution: derive problems from real delivery data
- Data bottleneck: compute
  - Solution: efficient data collection
  - Parallelize both environment (VRP simulation) & policy (neural network)
- Reward shaping
  - Vehicle penalty = sparse reward
  - Penalty may come hundreds of steps later
  - Solution: penalize *unused capacity*



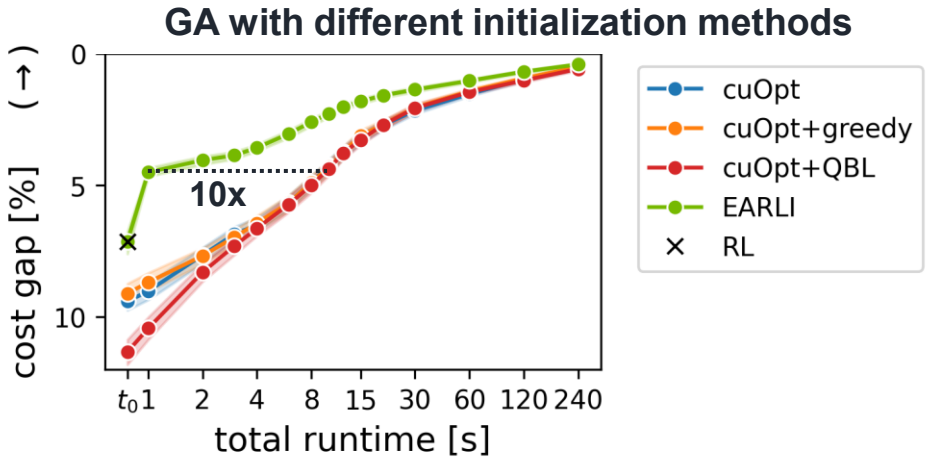
Real delivery data in Rio de Janeiro



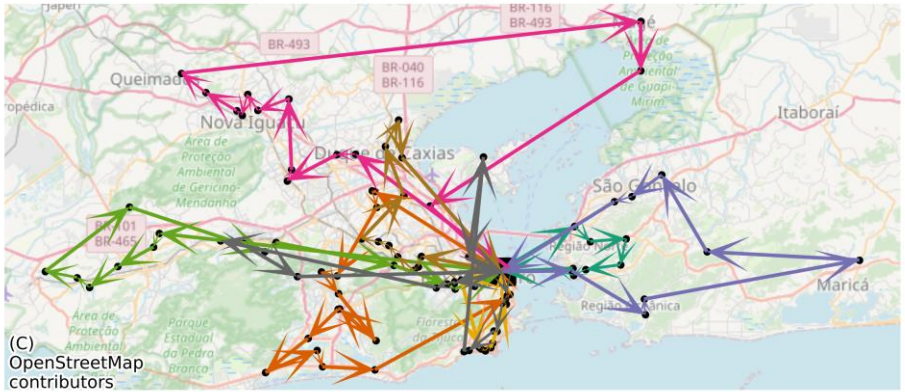
Capacity penalty improves vehicles-vs-cost tradeoff

EARLI: · Same costs – much faster

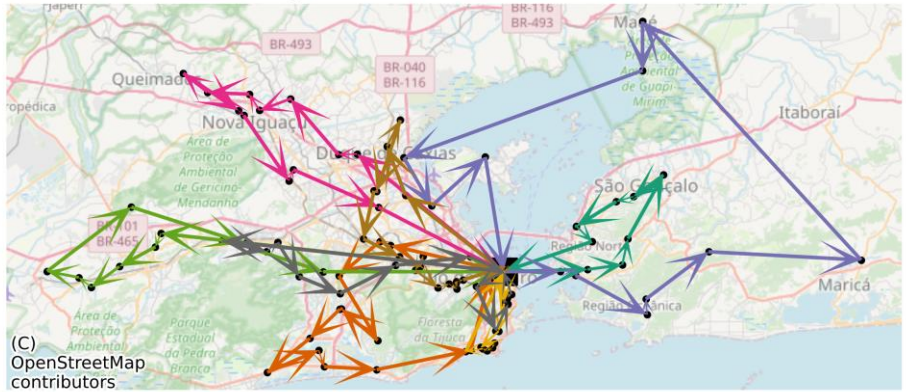
- 256 instances X 500 customers in Brazil
- Accelerating SOTA solvers
- Code available on GitHub



cuOpt: cost=67309

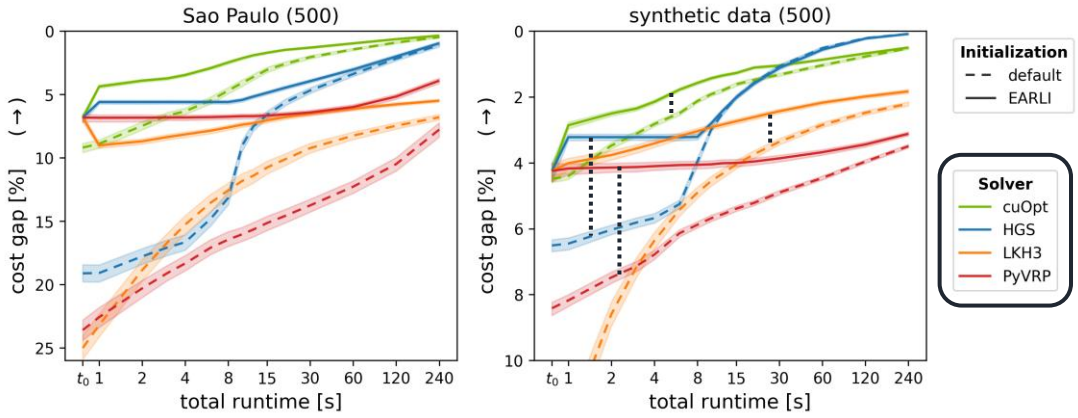


cuOpt+EARLI: cost=65258

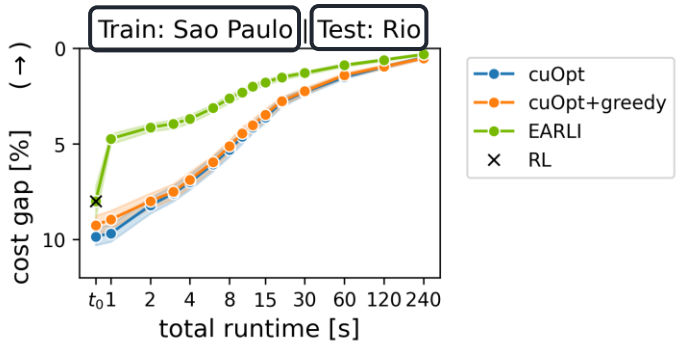


EARLI: · Same costs – much faster

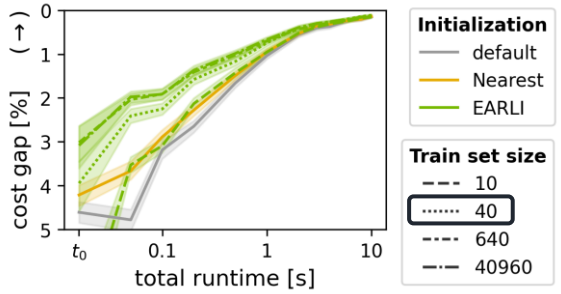
EARLI improves various solvers beyond cuOpt



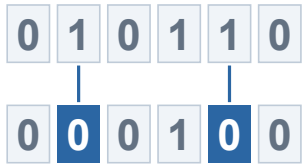
EARLI generalizes to new distributions



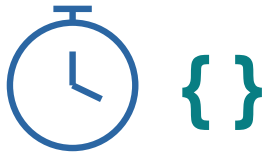
EARLI can learn even with few training instances



Optimization is a great test-bed for ideas



An action is flipping a few bits



Fast, easy to implement

1% › $

Immense economic value

Fundamental questions

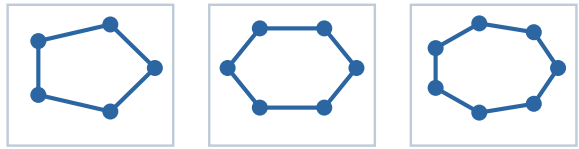
1 How to incorporate time / compute into the ML formalism?

$$\mathcal{L}(\theta) = \frac{1}{N}\sum_{i=1}^{N} \ell(f_\theta(x_i), y_i) + \lambda\,\Omega(\theta)$$

+



2 **Solved NP-hard problems**



Generalization

Polynomial average time?